Counting Versus Testing the Destination: A Simulation Study of Layered Defences Against SMS OTP Flooding and Pumping

Naveed Ahmed[a,*]

[a]College of Computer and Information Science, Prince Sultan University, Riyadh, Saudi Arabia

Abstract

Endpoints that send one-time passwords via SMS must be reachable before any account exists, and they are abused to waste a service's messaging budget (flooding) or to earn part of the termination fee on a colluding operator's number ranges (SMS pumping). Existing defenses limit the request source, score each request with labeled traffic, or block destination prefixes, but a service that sees only its own traffic has no controlled evidence of which destination signal to act on, the number of sends to a block or the fate of their codes, nor of what either costs the legitimate users who share a block with an attacker. We evaluate in simulation a twelve-step defense for a registration flow, which draws trust from app attestation, signed sessions and verification outcomes, against a seven-control baseline, five simpler designs and 25 attacker profiles, and compare the two label-free destination policies, a counter of sends per number block and sequential tests of code outcomes, under two protocols we committed before their runs. Our results show that attacks carrying a signature in the request are contained at a low cost to simulated users, whereas a residential attacker with farmed CAPTCHA scores is only rationed, by refusing half of first-time users. On the destination, short-window counters of SMS sent per number block, selected at a benign service target, leaked 6 to 48 messages against four concentrated 20-minute pumpers at 0.06 to 0.43 points of attack-free completion, where the default sequential tests of verification outcomes leaked 19 to 452 at the busiest density; on seeds and workloads held out from selection, the frozen counter met its leakage and benign-cost claims and leaked no more than the tests in all but 13 of 144 cells, all against an instantly verifying carrier, but failed both service-under-attack claims when the pumper shared blocks with real users. Unfitted estimates ordered the policies' mean leakage in 18 of 18 decisive boundary cells with the counter's cold-start bound and 17 with its warm-start bound; their crossing is algebraic, not a measured switching point, and the counter loses its advantage against paced or widely spread pumpers. These findings rest on simulated users and assumed calibration. For operators they give a tested ordering of destination policies against short, concentrated pumping, and identify the quota per block, and the competition for it on a shared block, as what decides a counter's cost.

Keywords: SMS pumping, Artificially inflated traffic, One-time password, Rate limiting, Sequential tests, Fraud prevention

1. Introduction

Most online services verify a phone number by sending a code via SMS from an endpoint that must be reachable before any account exists, at a tenth of a cent to twenty cents per message. Two abuses follow from that design. A flooder makes the service send to numbers it does not own, so the service pays for messages nobody asked for; a pumper sends to a range whose operator shares the termination fee, so the service's SMS budget becomes the attacker's income, a technique that MITRE catalogues as resource hijacking (MITRE Corporation, 2026). Pumping is the SMS form of international revenue-share fraud, whose voice form was estimated at $10.76 billion a year in 2015 (Sahin et al., 2017). The sums at a single service are not small: Twitter's owner said in 2023 that pumping had cost the company approximately $60 million a year (Commsrisk, 2023), and Huh et al. (2025) found as many attack requests as genuine ones in three months of one large service's authentication logs.

Defenses act at three points on the path of a request. At the source, services limit requests per address, session and device, challenge them with CAPTCHAs, block known proxies and restrict destination countries, and risk-based authentication combines such signals into a score with graded responses (Wiefling et al., 2019). At the messaging vendor, Twilio's Fraud Guard blocks destination prefixes and countries whose traffic departs from an account's history (Twilio Inc., 2026c), and its per-number pumping risk score advises friction from 60 and refusal above 90 (Twilio Inc., 2026a). On a service's own logs, Huh et al. (2025) detect attack requests one at a time with gradient-boosted trees, at 89.6% recall and 0.19% false positives on web traffic, and Zaliskyi and Odarchenko (2026) count application-to-person sends per number prefix in five-minute windows and quarantine a prefix that crosses a threshold.

Each of these rests on an assumption that the literature has tested and found wanting. Source-side limits assume that an identity is scarce; without a trusted authority, one entity can present as many identities as it likes (Douceur, 2002). Addresses are not scarce either: five residential proxy services relayed through 6.2 million addresses in 238 countries, 90% of

*Corresponding author.
Email address: nahmed@psu.edu.sa (Naveed Ahmed)

them for under fifteen minutes each and 2.2% of them on any blacklist (Mi et al., 2019), and carrier-grade NAT, deployed in more than 90% of cellular networks, puts dozens of subscribers behind one address and makes address reputation "all but infeasible" (Richter et al., 2016). A CAPTCHA costs an attacker about a dollar per thousand solutions from a labor market that was already paying ten dollars per thousand in 2007 (Motoyama et al., 2010). Supervised detection needs labels, which Huh et al. (2025) derived from traffic peaks after the fact, and a decision taken per request can be probed one threshold at a time. Prefix counting acts on the one key a pumper cannot rotate away from, since it is paid only on its partner's ranges, but the thresholds that stopped 95% and 63% of two incidents in the data of Zaliskyi and Odarchenko (2026) also blocked 58% of a legitimate marketing campaign.

The two closest studies therefore each act on one destination signal, and neither measures what acting on it costs. Huh et al. (2025) report that about 92% of attack requests fell on prefixes with zero conversion, against 21% for genuine traffic, so whether codes are entered is informative; but the signal enters a classifier that needs labeled traffic and was not released because it is deployed. Zaliskyi and Odarchenko (2026) act on send counts alone, without outcomes, and conclude that counts by themselves cannot safely separate inflated traffic from a campaign. Neither reports what a destination policy costs the legitimate users who share a number block with the attacker, neither reports its cost when there is no attack, and neither fixed the setting it would select before seeing the evaluation traffic. A service that sees only its own traffic is thus left without controlled evidence on the question it can act on: whether to count sends per block or to test the outcomes of their codes, at what quota or threshold, and at what price to its own users.

We make that comparison in simulation, inside a complete defense for a registration flow. We start from a baseline of seven controls, v1, and show how an attacker within a stated threat model breaks each of eight assumptions behind them. Our twelve-step redesign takes the platform from an attestation rather than a header, binds requests to signed sessions, checks the number before a message is paid for, builds reputation from whether earlier codes were entered, scores each request into one of five responses, and enforces a hard budget (Figure 4 and Algorithm 2); on the destination it offers two label-free policies keyed on the 8-digit number block, a short-window counter of sends and sequential tests of code outcomes. We run seven designs and 25 scripted attacker profiles, from a single address to pumpers that enter codes, fake delivery receipts or buy challenge solutions, over simulated users on a simulated clock (Figure 5). We committed two protocols before their runs: the first selects destination settings on tuning seeds at a benign service target, and the second, written after that selection, freezes the selected counter and tests five claims on seeds and workloads held out from selection. We release the implementation, the protocols and every run's configuration and outcome (Ahmed, 2026). Everything is simulated: three behavioral parameters have published counterparts and the rest are our assumptions, so the results describe our generator rather than real traffic.

We ask one question in three parts. On the source side, how much of the flooding that a seven-control baseline admits does a twelve-step redesign contain, at what cost in completed registrations, and through which layer? Where identities are cheap, when does reputation built from verification outcomes stop separating an attacker from the users whose keys it shares, and what does rationing then cost and depend on? On the destination, where a pumper is paid, does counting sends per number block or testing the outcomes of their codes leak less, under which conditions does the answer reverse, and what does either policy cost the legitimate users who share a block, with and without an attack? We make four contributions.

- **A design, as pseudocode and as code.** We show how an attacker breaks eight assumptions behind a seven-control baseline (Section 3.1), close the gaps with a twelve-step pipeline whose trust comes from attestation, signed sessions and the fate of earlier codes (Figure 4, Algorithm 2), and release the implementation with a regression test for every replay and concurrency defect we found on the way.

- **The limits of source-side reputation.** We measure where reputation built from code outcomes stops separating a residential attacker from the users whose keys it shares, and what rationing then costs: the default cap holds a CAPTCHA farm to 46% of its requests by refusing 54% of first-time users (Sections 5.3–5.4).

- **Counting against testing, under predeclared protocols.** We compare a short-window counter of sends per number block with sequential tests of code outcomes, selected at a stated service target, then frozen and tested on held-out seeds and workloads, and charge each policy with its leakage, its attack-free cost and its harm to the users who share a block with the attacker (Sections 5.5–5.7).

- **Unfitted leakage estimates.** From the rules and the attack rate alone we derive when counting leaks less than testing, and check the estimates against measurements over attacks of up to an hour: they ordered the policies' mean leakage in 18 of 18 decisive cells with the cold-start bound and 17 with the warm-start bound (Section 3.7, Figures 9 and 10).

Section 2 reviews related work; Section 3 gives the baseline, the threat model and our design with its leakage estimates; Section 4 our methodology; Section 5 reports the results; Section 6 discusses them, sets the design against the benchmark designs and the nearest work, and states the limitations; Section 7 concludes.

2. Related work

We group prior work by where it acts on the path of an OTP request, which Figure 1 draws. The attacker's bots, working through a botnet or behind residential proxies, open the target website and ask it for a code; the website's service endpoint receives each OTP request and decides whether to send; an SMS vendor routes the message; and a terminating operator delivers it, in a pumping attack to premium-rate numbers that a number

provider has placed with that operator. Two flows run against the direction of the request. The outcome signal, whether the code was entered or not, returns from the website to the service endpoint, and it is what one of our two destination policies uses. The revenue-sharing line, drawn dashed, carries part of the termination fee from the operator through the provider of the premium-rate numbers back to the attacker, which is why the destination is the one key a pumper cannot change. The works we review sit at one of these points. At the client, the bots and the proxies, are the measurements of what an attacker can buy: residential proxy addresses (Mi et al., 2019), shared addresses behind carrier-grade NAT (Richter et al., 2016), CAPTCHA solutions (Motoyama et al., 2010; Dionysiou and Athanasopoulos, 2020) and as many identities as it likes (Douceur, 2002). At the service are the controls an endpoint can run itself: per-entity limits, CAPTCHA, proxy and country lists (our v1 baseline, Section 3.1), risk-based scoring of logins (Wiefling et al., 2019) and supervised per-request detection on labeled logs (Huh et al., 2025); our twelve source-side steps and two destination policies act here too (Sections 3.3–3.5). At the vendor are prefix and country blocking from an account's history and a per-number pumping risk score (Twilio Inc., 2026c,a), and counts of sends per prefix in five-minute windows with a 24-hour quarantine (Zaliskyi and Odarchenko, 2026). At the operator and the number provider, where the money is made, are the revenue-share number ranges that providers advertise and share, and detection from call records and test-number intelligence (Sahin et al., 2017; Sahin and Francillon, 2021). We close with the gap we address (Table 1).

2.1. *Revenue-share fraud*

In international revenue-share fraud, a fraudster obtains "pseudo" premium-rate numbers from websites that pay for traffic and generates calls to them; a transit operator short-stops the calls, keeps the termination fee and shares it with the fraudster. The taxonomy of Sahin et al. (2017) sets the scheme among frauds that persist because call routing is opaque, each operator knowing only the next hop, and because operators are reluctant to share pricing, routes or fraud findings with competitors. Sahin and Francillon (2021) then measured the market from the supply side, collecting 3,149,793 distinct test numbers that twelve such providers advertised between January 2016 and November 2019 across 238 destinations. The ranges are a managed asset: 80% of them were shared across providers, which subdivide a hijacked block among themselves, and 70% lay inside valid numbering plans. From this intelligence and from call histories, the authors built a random-forest detector for an operator's call records that reached 98% accuracy at 0.28% false positives, where blocking the two-digit neighborhood of a known test number caught 63%. Both studies concern voice telephony, and the detector needs the test-number intelligence and the call histories of an operator. The lesson for SMS is that the number range, not the calling identity, is what the fraudster must keep, and that a party seeing only its own traffic detects the fraud late. An OTP endpoint is in a weaker position still: it holds neither intelligence nor call histories, only its own sends and whether their codes were entered. That is why the policies of Section 3.5 are keyed on the 8-digit block and use only those two quantities.

2.2. *Detecting inflated SMS traffic*

Huh et al. (2025) trained gradient-boosted trees on 9.4 million SMS authentication logs from nineteen countries, reaching 89.6% recall at 0.19% false positives on web traffic and 91.1% at 0.11% on app traffic. The logs came unlabeled from one large service over three months of 2023; the authors labeled attacks inside traffic peaks, as web identities with four or more requests and no successful validation in a day, or app number-and-device pairs never validated, and treated requests outside peaks as genuine. Two models, with 18 and 13 features at three

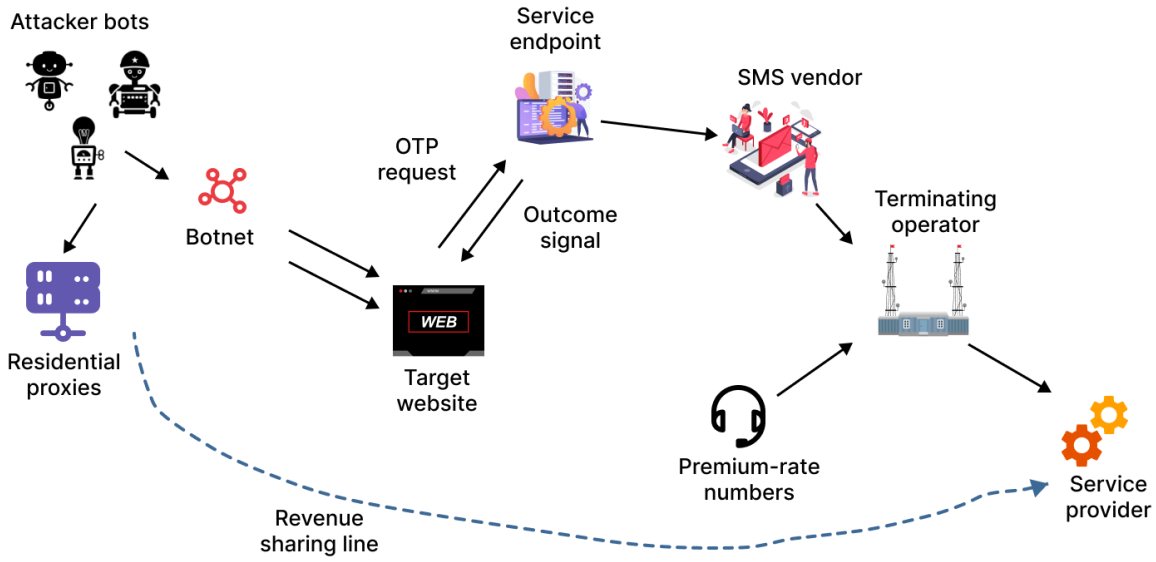


Fig. 1. The path of an OTP request, and where the money goes.

levels (the request, the identity's history, and country-wide aggregates over 24 hours), run per request before the message is sent. Approximately 73% of web attacks sent four or fewer requests per identity a day, defeating per-entity limits, and destination signals were the strongest: about 92% zero conversion for attacks against 21% for genuine traffic, and 91% of app attacks fell on prefixes with more than 30 distinct numbers in a day against 0.01% of genuine requests. The same data show the counter-move: in three countries, about 82% of attack requests validated their codes, which the authors attribute to attackers avoiding conversion-rate rules. Their detector requires labeled traffic and acts per request; the cost to legitimate users is reported as false positives only, and the model was not released because it is deployed. Their data show that destination aggregation is informative; we add a controlled comparison of two label-free ways to act on it per block, from send counts or code outcomes, and of their cost to users sharing a block. We did not reproduce theirs, so we do not rank the two.

Zaliskyi and Odarchenko (2026) count A2P sends per prefix (a number less its last three digits) in fixed or rolling five-minute windows and quarantine a prefix for 24 hours past a day or night threshold. On metadata of two confirmed incidents fixed windows blocked 95% and 63% of the traffic, and 58% of a legitimate marketing campaign; counting unique recipients blocked 2% of the campaign and none of the first incident. The authors conclude that message counts alone cannot safely separate inflated traffic from a campaign; they do not use verification outcomes. Theirs is a send counter judged on real traffic by messages blocked; ours compares a counter with outcome tests at a service target, in simulation, by what users sharing a block lose; the numbers are not comparable.

The messaging vendors act one hop later. Twilio's Fraud Guard analyzes an account's historical traffic and blocks sends to a destination prefix or country whose pattern departs from it, with three protection levels and claimed false-positive rates under 1% and 2% (Twilio Inc., 2026c); its SMS Pumping Risk Score rates a number from 0 to 100 from Fraud Guard, risky carriers, traffic patterns and low conversion, and advises friction from 60 and refusal above 90 (Twilio Inc., 2026a). Graded destination action is therefore established practice, but the signals and models are undisclosed, so we benchmarked no product.

Taken together, these works establish that attacks concentrate on destination prefixes and avoid converting, that counts alone misfire on campaigns, and that some campaigns now enter codes to pass conversion rules. None of them compares counting with outcome testing on the same traffic, fixes its setting before the evaluation runs, or charges a policy with the registrations it costs the users of a shared block. Those are the questions of Sections 5.5–5.7, and we modeled the attackers that enter codes, fake receipts and build trust (Table 2) on the counter-move above.

2.3. What an attacker can buy

What a source-side control costs an attacker has been measured. Motoyama et al. (2010) bought CAPTCHA solutions from eight retail services at $1 to $20 per thousand, found that the wage for solving had fallen from $10 per thousand in 2007 to about a dollar by 2009, and measured a median response of 14 seconds at error rates of 10% to 20%; human labor had displaced automated solvers, and the authors conclude that a CAPTCHA limits access to attackers whose business is profitable despite that cost rather than keeping machines out. Dionysiou and Athanasopoulos (2020) classify the machine-learning solvers that have since appeared. Addresses have a market too: Mi et al. (2019) infiltrated five residential proxy providers and observed 6,183,876 addresses in 238 countries and 52,905 ISPs, 95% of them residential, 90% relaying for under 870 seconds and 2.2% on any blacklist; Richter et al. (2016) found carrier-grade NAT in more than 90% of cellular networks, with 64 subscribers behind one address where each receives a thousand ports, and conclude that address reputation and blacklisting become "all but infeasible" there. Behind both markets lies the older observation of Douceur (2002) that without a trusted authority one entity can present many identities, so a signed session provides continuity, not scarcity. Risk-based authentication, which Wiefling et al. (2019) probed at eight large services, scores a login from its address, user agent, resolution, time and similar features into risk buckets and asks for a further factor above a threshold; every service that showed such behavior used the address, and none disclosed its procedure. Step 7 of our design resembles it, but scores a request made before any account exists, so the account history that risk-based authentication relies on is absent. Client puzzles (Dean and Stubblefield, 2001) would add a computational price per request; we did not evaluate them.

These measurements say what an attacker can buy. For a defense they imply that per-identity controls tax identity reset rather than prevent it, and that the question for a residential attacker is not whether it can be told from a person but what rationing it costs the people it resembles. Sections 5.3 and 5.4 measure that in this design.

2.4. Sequential tests

Our block tests are Wald's sequential tests (Wald, 1945) run as Page's cumulative sums (Page, 1954), with a floor parameter that selects between the two (Section 3.5). Quickest change detection minimizes detection delay under false-alarm constraints (Veeravalli and Banerjee, 2012), and a CUSUM's parameters follow from the error rates it must meet (NIST/SEMATECH, 2026); windowed statistics and the time-uniform confidence sequences of Howard et al. (2021), which stay valid at any stopping time, are not evaluated here, so 'testing' below means our conversion and speed tests only, and we report their error rates empirically rather than derive them.

2.5. Attacks on the delivered code

Malware on the handset or a social-engineering call can take a code after it is delivered (Mulliner et al., 2013; Siadati et al., 2017). Those attacks begin where our work stops: we ask whether a message should be sent at all, and leave the protection of a delivered code aside.

2.6. The gap

Table 1 compares the works closest to ours on the evidence they use, the signal they act on, where they act, and whether they measure the cost to legitimate users. Across them, the destination is agreed to be where a pumper is exposed, and three ways to act on it are in use: a supervised model with labels, a count per prefix, and a vendor's undisclosed score. What none supplies is a controlled comparison, under a protocol fixed before the runs, of the two signals a service can compute without labels, with the cost to the users who share a block made part of the result. We supply that comparison, in simulation, together with the source-side layers it sits in.

3. Threat model and design

This section describes the baseline we replace, the attacker our design must resist, the twelve steps and two destination policies we designed, and the estimates of leakage we derive from its rules.

3.1. The baseline and its gaps

The baseline, v1, is our starting point, the registration endpoint as it stood before this work: it sends a code to any unregistered number once seven controls pass, in the order of Algorithm 1. A request from a known VPN or proxy address is refused. The platform is read from a client header, and a request that claims iOS or Android skips the two checks that web requests face, a reCAPTCHA v3 score threshold and an origin check. The number's country must be on an allow-list unless the request carries the bulk flag; the body must be within a length limit and the number not on an exclusion list unless the request comes from a test server. One message per number per minute, five requests per address per minute and static caps per platform and country complete the controls. Each cap is a counter that is read when the check runs and incremented after every check has passed, and the response names the control that refused.

Each control rests on an assumption that an attacker within the threat model of Section 3.2 breaks. We derived the eight gaps below from the rules, not from observed abuse; Algorithm 1 marks where each opens. (A) The platform exemptions trust a client claim, so any request can send the header for iOS and skip the reCAPTCHA and origin checks. (B) Five requests per address assume that addresses are scarce; a residential proxy pool supplies a fresh address per request, nothing throttles a subnet or an autonomous system, and no client identity survives an address change. (C) A number is sent to without validation, so numbers that cannot exist, numbers on dead ranges and generated sequences cost the same as a real message. (D) The country allow-list is the only notion of destination cost; premium-rate and revenue-share prefixes inside an allowed country cost many times a standard message, and a pumper targets exactly those. (E) Every limit is per key, so a wide attack that stays under each cap burns the budget without bound; nothing watches total volume or spend. (F) A limit checks its counter and increments it as two operations, so two concurrent requests on different instances both pass. (G) The bulk and test paths skip checks but need no stronger authentication than a normal request, so a leaked flag is a bypass. (H) Every decision is pass or fail and the response names the control that refused, so an attacker can probe each threshold, stay under it, and enumerate registered numbers.

3.2. Threat model

The attacker is a party that wants the service to send as many messages as possible, or as many as possible to numbers that pay it. The first is a flooder: the service pays per message, and the attacker's gain is the service's loss. The second is a pumper, paid a share of the termination fee by the operator that terminates its ranges; it earns nothing on any other number, so its traffic concentrates on the blocks it is paid for. The share

Table 1
The closest work, set beside ours.

Work	Evidence	Signal acted on	Where it acts	Cost to legitimate users	Left open for an OTP service
Sahin and Francillon (2021)	Operator call records; 3.1 million test numbers advertised by 12 providers, 2016–2019	Distance of the called number to advertised ranges and call history; labels from matched test numbers	Per call, at an operator	A false-positive rate (0.28%), not harm to callers	Voice calls; needs number intelligence and call histories an OTP endpoint does not hold
Huh et al. (2025)	One service's 9.4 million SMS authentication logs, 19 countries, three months	Per-request features with prefix-level counts and conversion; labels derived from traffic peaks	Per request at the service, before the message is sent	False positives only (0.19% web, 0.11% app)	Needs labels; the deployed model was not released; a per-request decision can be probed
Zaliskyi and Odarchenko (2026)	Aggregator delivery metadata: two confirmed incidents and one legitimate campaign	Sends, or unique recipients, per prefix in five-minute windows; no labels	Per prefix, with a 24-hour quarantine	Share of a campaign blocked (58%; 2% with unique recipients)	Outcomes unused; thresholds that stop incidents block campaigns; no per-user cost
Twilio Inc. (2026a,c)	Vendor-wide traffic across accounts; documentation only	Departure from an account's history per prefix and country; a per-number score from carriers, patterns and conversion	Before the send, at the vendor; bands at 60, 75 and 90	Claimed false-positive rates under 1% and 2%	Signals and models undisclosed; cannot be benchmarked or reproduced
Wiefling et al. (2019)	Black-box tests of eight services with 224 accounts	Address, user agent, resolution, time and similar login features scored into risk buckets	Per login to an existing account	Not measured; re-authentication observed	Scores a login, not a request made before any account exists; procedures undisclosed
This study	Simulation: 25 attacker profiles, seven designs, two predeclared protocols; code and every run public	Sends per number block; sequential tests of code outcomes per block; no labels	Per 10 000-number block at the service, before the message is sent	Attack-free benign cost and extra harm under attack, per cohort, in completed registrations	Simulated users and assumed calibration; no real traffic replayed

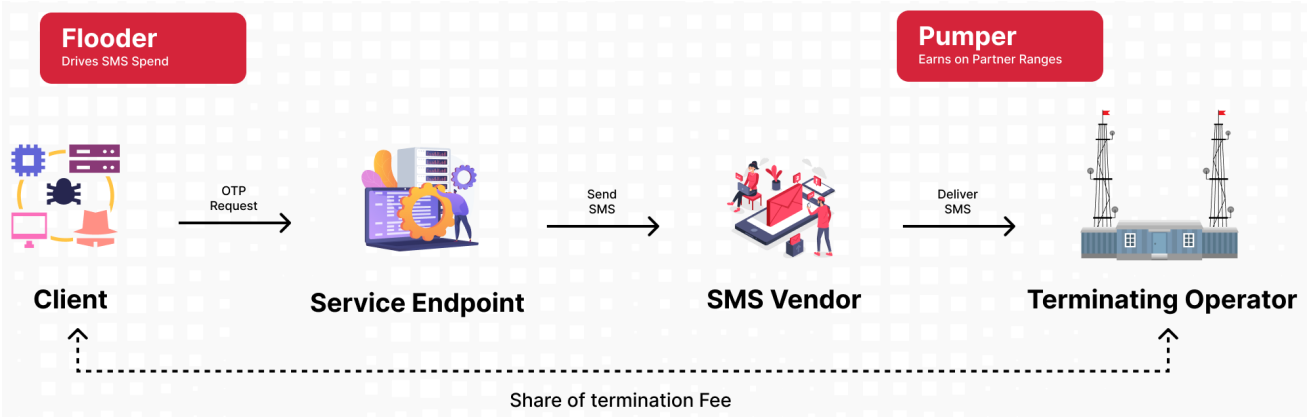


Fig. 2. The two attackers on the path, and the fee that flows back to one of them.

Algorithm 1 The v1 baseline; each comment names the gap the rule above it opens.

Input: a request r: address, platform header, number, body, bulk and test flags

1: **if** r.address is a known VPN or proxy address **then refuse**
2: $p \leftarrow r$.header[`HTTP_PLATFORM`]
▷ *A: the platform is a client claim; a header saying iOS or Android skips lines 4–5*
3: **if** p = web **then**
4: **if** reCAPTCHA v3 score(r) < 0.5 **then refuse**
5: **if** origin(r) $\notin$ allowed domains **then refuse**
6: **if not** r.bulk **and** country(r.number) $\notin$ allow-list **then refuse**
▷ *D: premium ranges inside an allowed country pass; G: the bulk flag needs no credential*
7: **if not** r.test **and** (body too long **or** number excluded) **then refuse**
▷ *G: the test flag needs no credential*
8: **if** sends(r.number, 1 min) $\geq$ 1 **then refuse**
9: **if** requests(r.address, 1 min) $\geq$ 5 **then refuse**
▷ *B: a fresh address per request; nothing throttles a subnet or an ASN*
10: **if** requests(p, country, window) $\geq$ static cap **then refuse**
▷ *E: every cap is per key; nothing bounds the total or the spend*
11: increment the counters read in lines 8–10
▷ *F: read, then write: two racing instances both pass*
12: send the SMS to r.number
▷ *C: the number is never validated or priced before it is paid for*
13: **return** the control that refused, or success
▷ *H: each threshold can be probed, and registered numbers enumerated*

is not public, so in Section 5.8 we report instead the share at which each scripted pumper breaks even. Figure 2 places the two attackers on the path of a request: the flooder, whose aim is the service's SMS spend, and the pumper, who earns on its partner's ranges through the share of the termination fee that the dashed line carries back from the operator to the client. What each controls at the four points of the path, what neither can do, and what the model leaves out follow.

At the client, the attacker runs scripts behind residential proxies in the service's own country, with a fresh address per request, or from a datacenter abroad, or spread at a low rate over twenty autonomous systems. It buys human-like reCAPTCHA v3 tokens from solving services at about 0.3 cents each, so a score threshold alone does not stop it. It mints a fresh browser fingerprint per request, pre-ages fingerprints for hours, or reuses one two-day-old profile; it can present a forged platform header; and it draws numbers at random, walks them upward, or takes them from its partner's blocks. It knows every threshold, the parameters of the destination tests and the timing of the workers, and it sees only the one uniform response that Section 3.3 describes.

On its own ranges, a pumper's operator can do what a real recipient could: enter the code it receives, within a second or after a human-like delay, or only when a block's statistic nears its threshold; report deliveries as failed, so that a billed send feeds no test; and verify on hundreds of identity-and-number pairs for ten or thirty minutes to build trusted history before flooding with the same pairs. It can spread over three to three hundred blocks, at full rate or paced to a counter's quota, and it can buy a solution for every interactive challenge.

Three things it cannot do. It cannot forge an app attestation, so a platform established at Step 0 is evidence rather than a claim. It cannot earn on a range its partner does not terminate, which is why the destination is the one key it cannot rotate away from. And it cannot make a stranger enter a code, so a code entered on a block that real users share is honest evidence about that block; what a code entered on the attacker's own number buys it, we measure rather than assume. Attacks on a code after delivery (Section 2.5) and compromise of the service, its vendors or the attestation providers are outside the model. Table 2 lists the 25 profiles that instantiate these capabilities; pool size, rate and CAPTCHA class are drawn per seed.

3.3. *The pipeline*

Figure 3 shows the implementation as deployed. An HTTP API, run on any number of instances, offers the client five public endpoints: an attestation challenge and an enrolment, a session endpoint where a reCAPTCHA token or an attestation buys a signed session, the request endpoint, which runs the pipeline and returns one uniform body, and the verify endpoint, which takes the code; two internal endpoints, behind a service credential, take the vendor's delivery receipts and drive the timeouts. Behind the API sit the two procedures of Algorithm 2: the pipeline, which runs Steps 0–11 on each request, and the feedback loop, which applies each transition of a send (delivered, verified, failed, undelivered) as one compare-and-set. All state

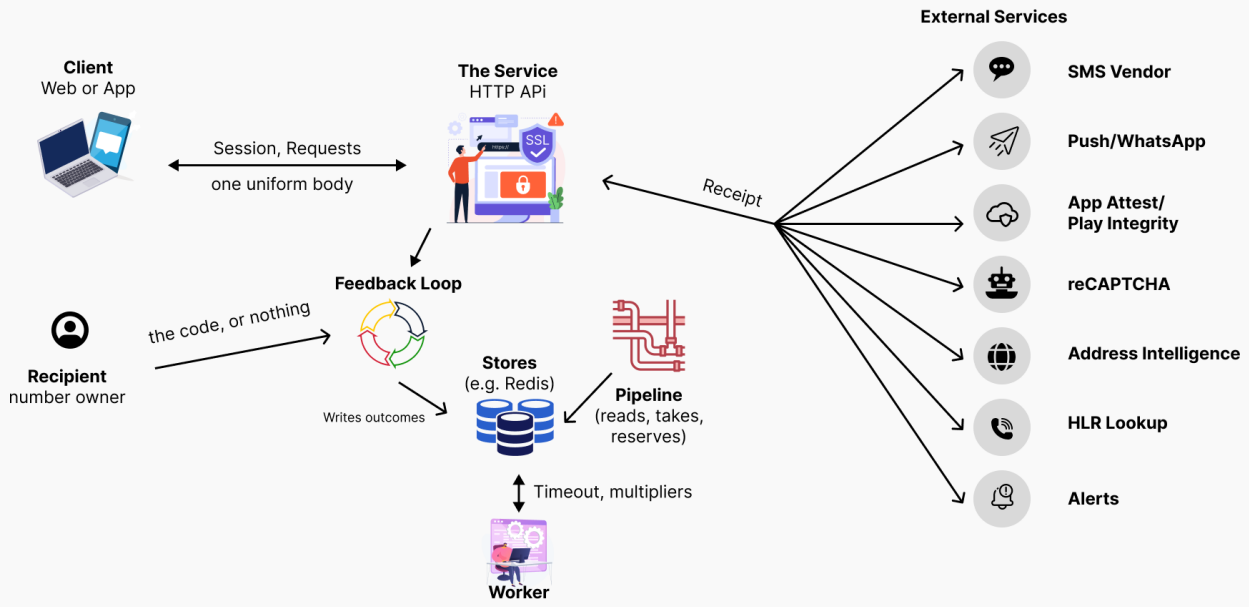


Fig. 3. How the implementation fits together.

lives in one store, Redis in production and an in-memory twin with the same rules in the tests and the simulator: the sessions, every limit and reservation, the SMS history, the reputation counts, the block statistics and counters, the denylist, the adaptive multipliers and the operating mode. What the loop writes, the next request reads, which is how the outcome of one send changes the judgment of the next, and why any instance can verify any code. One worker per deployment ticks every minute to resolve the sends that timed out and to run the baseline job that sets the multipliers of Step 9 from the pipeline's own counters. Every external dependency, attestation, reCAPTCHA, address intelligence, the HLR lookup, the SMS vendor and the downgrade channels, sits behind an adapter with an in-memory fake, and a deployment reports which adapters are fake so that production can refuse to start with any. The simulation of Section 4 runs the pipeline, the feedback loop, the worker's two jobs and the in-memory store as drawn, without the HTTP layer, replaces the external services with their fakes, and in the main study hands the baseline job the modeled legitimate rate instead of letting it learn it (Section 5.4 runs the learning job).

We designed the pipeline so that no message is paid for before every check passes, and cheap checks run first. Trust comes from evidence the client cannot forge, an attestation, a server-signed session, and the fate of earlier codes, though such evidence can be manufactured (Section 5.8).

A request passes the twelve steps of Figure 4 in order until one refuses it. The figure numbers its stages in reading order: its stages 1 to 4 are Steps 0 to 4 of the text and of Algorithm 2, with Steps 3 and 4 drawn as one stage, the web checks; from stage 5 on the two numberings coincide; and stage 12 is the feedback loop. The letters below are the gaps of Section 3.1 that each step closes. Step 0 establishes the platform from an App Attest or Play Integrity result and refuses proxy addresses (A). Step 1 binds the request to a server-signed session with a fingerprint and a replay nonce, three requests per session per ten minutes (B). Step 2 throttles the address, its /24 and its autonomous system, with lower caps for hosting networks (B, F). Steps 3 and 4, for web requests only, check the reCAPTCHA score and the origin. Step 5 prices and checks the number before anything is paid for: its country and cost class, sequential or narrow-range bursts, liveness through an HLR lookup, and the verdict stage of its block, with the bulk flag now needing an internal credential (C, D, G). Step 6 validates the body and the exclusion list, the test flag needing a credential too (G). Step 7 computes the risk score of Equation (1) and chooses one of five tiers (H). Step 8 applies a per-number backoff, sixty seconds doubling to an hour and five sends a day. Step 9 takes the source caps per source, platform and country, multiplied by the adaptive factor. Step 10 sets the operating mode from the hourly budget, elevated above 80% and emergency at 100% (E). Step 11 reserves the hourly count, the spend and any destination slot in one atomic operation, writes the state and enqueues the message (E, H). A refusal at any step returns one uniform reply, and every send resolves through the feedback loop, whose per-key and per-block outcomes are what the next request is judged on.

Algorithm 2 gives the same pipeline as we implemented it, with the feedback loop of Section 3.4 and the block statistics of Section 3.5 as its second procedure. Read against Algorithm 1, it differs in four ways: the platform is established before anything else and from evidence, every limit is taken atomically and released on refusal, the number is priced and checked before it is paid for, and the outcome of each send flows back into the keys and the destination block that the next request will be judged on. Each limit and reservation is one atomic Redis operation, but the enclosing steps are not one transaction: on the normal path, a refused request returns its per-number claims and counter slot, and the hourly budget is reserved only

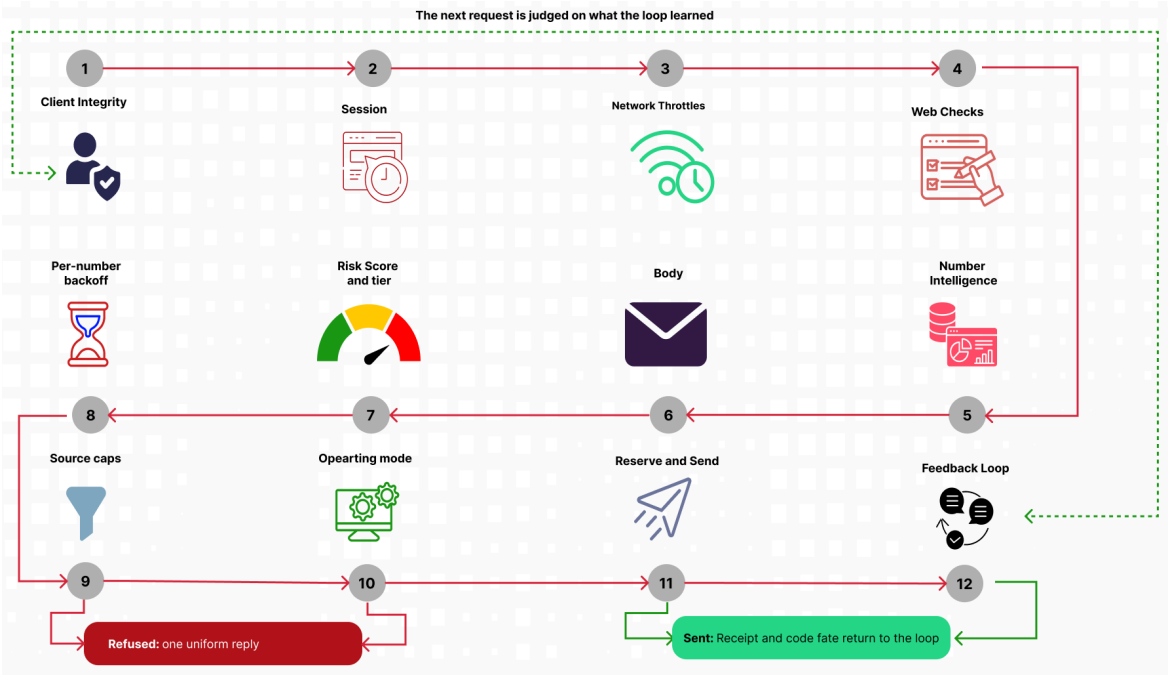


Fig. 4. The twelve steps, and the loop that feeds the next request.

for a message about to be enqueued, a reservation rather than a completed dispatch; a crash between them can leak reservations (Section 4). Address limits and source caps count requests, except that a request Step 11 challenges returns its source-cap count. Every outcome after Step 0, except the challenge, returns one HTTP 200 body after at least 400 ms, a floor that cannot hide longer work.

Step 7 computes

$$s = \sum_i w_i x_i + w_c \frac{\max(0, \theta - r_{\min})}{\theta} + w_s f + w_m i - w_v v_n - w_f v_f, \quad (1)$$

where the x_i are request signals (CAPTCHA score, hosting ASN, fingerprint age, session requests, country mismatch, number class), $r_{\min}$ is the lowest conversion ratio over the request's keys (address to destination block), $\theta = 0.3$, $w_c = 25$, f and i flag a flooded key and one whose codes are entered within seconds, and v_n, v_f mark a number and fingerprint with verified history; other weights are 5 to 25 (Ahmed, 2026). Tiers start at 20, 40, 60, and 80: *allow*, *delay* (a doubling wait), *challenge* (−20 when solved), *downgrade* (push or WhatsApp), and *block*.

3.4. The feedback loop

A send resolves as *verified* when its code is entered, *fails* after five incorrect attempts or two minutes past the delivery receipt, and is *undelivered* without a receipt in 60 s or with a failed receipt; only the first two count toward reputation. A positive receipt after a negative one reopens the send; a late verification reclassifies it, but an issued verdict stands. Each transition records its effects and its time in the same compare-and-set, and a recovery sweep completes the work of a dead process; each effect type applies at most once, and exactly once if the sweep runs within 20 minutes. Reputation counts land in the transition's hour, and a block keeps its latest outage failure time; however, a block's statistic accumulates in processing order, and a verdict is dated at the event that completes the crossing; thus, a late event can backdate a verdict that contains later evidence, moving its expiry and the next test's start (by 10 min in one diagnostic). Simulated runs apply events in time order, but concurrent live instances may not, because a transition is committed before its effects apply; thus, the same dependence holds live unless a block's effects are serialized.

3.5. Destination policies

A pumper is confined to its partner's ranges but can rotate numbers and blocks within them. Destination policies are keyed on the 8-digit block (10 000 numbers); therefore, protection weakens as a pumper spreads.

3.5.1. Sequential tests

The conversion statistic assumes that users verify with $p_\ell = 0.8$ and a flooder with $p_a = 0.1$: a failure adds $\ln \frac{1-p_a}{1-p_\ell} = 1.50$, a verification $\ln \frac{p_a}{p_\ell} = -2.08$. The speed statistic assumes that a machine enters a verified code within five seconds with $q_a = 0.9$ and a person with $q_\ell = 0.2$, because autofill removes the keystroke (Google LLC, 2026; Web Incubator Community Group, 2026); it adds 1.50 or −2.08. A verdict is issued, and the statistic is reset when either exceeds $h = \ln 1000 = 6.91$. The floor $S_t = \max(-ch, S_{t-1} + \text{increment})$ sets the memory: unbounded c is Wald's test, $c = 0$ Page's cumulative sum; the default is $c = 1$. A first verdict makes the block's first-time clients pass a challenge for an hour; a second within the hour moves them off SMS, refusing those without another channel.

Algorithm 2 The v2 pipeline and its feedback loop, as implemented.

procedure ProcessRequest(r) ▷ every refusal releases the claims taken so far
  $p \leftarrow$ Attest(r); **if** r.address is a proxy **or** p invalid **then refuse**
  ▷ *Step 0: App Attest or Play Integrity; the platform header is ignored*
  if session(r) unsigned, not bound to r.fingerprint, nonce replayed, **or** 3 requests in 10 min **then refuse**
  ▷ *Step 1*
  if r.fingerprint **or** r.address is denylisted **then refuse**
  for $k \in$ {address, /24, ASN} **do if not** AtomicTake(limit$_k$) **then refuse**
  ▷ *Step 2: one Redis operation per limit; lower caps for hosting ASNs*
  if p = web **and** (reCAPTCHA score $< \theta_r$ **or** origin $\notin$ allowed) **then refuse**
  ▷ *Steps 3–4*
  $m \leftarrow$ Normalize(r.number); **if** country(m) $\notin$ allow-list **then refuse**
  signals += cost class of prefix(m) (premium: **refuse**), sequential or narrow-range burst, VoIP; **if** HLR says dead **then refuse**
  $v \leftarrow$ BlockVerdict(block(m)) ▷ the stage in force on the 8-digit block
  ▷ *Step 5: the bulk flag needs an internal credential*
  if body invalid **or** m excluded **then refuse** ▷ Step 6: the test flag needs a credential
  $s \leftarrow$ Equation (1); $t \leftarrow$ Tier(s, mode) ▷ Step 7: allow, delay, challenge, downgrade, block
  if r has no verified history: $t \leftarrow \max(t, \text{challenge})$ if $v = 1$, $\max(t, \text{downgrade})$ if $v = 2$
  if t = block **then refuse**; **if** t = challenge **and** p = web **then return** challenge
  ▷ *the client retries with the solved challenge, which scores −20*
  if not AtomicClaim(m: window 60 s $\cdot 2^{\text{sends}}$ up to 1 h; 5 a day) **then refuse** ▷ Step 8
  $\mu \leftarrow$ AdaptiveMultiplier(source, p, country) $\in [0.25, 1.5]$ ▷ Step 9: from the baseline job
  if not AtomicTake(source caps $\times\mu$) **then refuse** ▷ verified history keeps the static cap
  mode $\leftarrow$ OperatingMode(hourly count, spend) ▷ Step 10: elevated at 80%, emergency at 100%
  if not AtomicReserve(hourly count, spend, block(m)'s counter slot) **then refuse** ▷ Step 11
  if the slot is past the quota **and** r has no verified history: $t \leftarrow \max(t, \text{challenge})$; past twice: $\max(t, \text{downgrade})$
  channel $\leftarrow$ SMS, or push or WhatsApp if t = downgrade; write state; enqueue(m, channel, delay if t = delay)
  return one uniform HTTP 200 body after at least 400 ms ▷ the same body for sent, dropped and refused

procedure OnOutcome(send, outcome) ▷ asynchronous; outcome $\in$ {verified, failed, undelivered}
  record the transition and its effects in one compare-and-set; apply each effect at most once
  ▷ *failed: five wrong codes or 2 min after delivery; undelivered: no receipt in 60 s, or a failed receipt*
  for $k \in$ keys(send) **do** ▷ address, /24, ASN, fingerprint, session, country, prefix, number
    verified$_k$ or failed$_k$ += 1 ▷ undelivered counts toward neither
    if k is a fine key, resolved$_k \geq 50$ **and** verified$_k$/resolved$_k < 0.1$ **then** denylist k for 24 h
  if verified **then** mark the number trusted ▷ verified history exempts the client from both destination policies
  $b \leftarrow$ block(number); conv$_b \leftarrow \max(-ch,\ \text{conv}_b + (\text{verified}\ ?\ -2.08 : +1.50))$
  if verified **then** speed$_b \leftarrow \max(-ch,\ \text{speed}_b + (\text{entered within 5 s}\ ?\ +1.50 : -2.08))$
  if conv$_b > h$ (after 3 events) **or** speed$_b > h$ **then** issue a verdict for 1 h and reset both statistics
  ▷ *stage 1, or stage 2 if a verdict is active: Section 3.5; $h = \ln 1000$, c the floor parameter*

3.5.2. Counters

A counter limits the number of SMS sent per block in a daily or short refilling window. Step 11 reserves a slot and decides the stage in one atomic operation when an SMS is about to be sent, so concurrent requests cannot skip the challenge. Past the limit, a graded counter challenges first-time clients, and past twice the limit moves them off SMS; a refusing counter refuses. Clients with verified history are exempt from both policies; therefore, a code entered on a pumper's number can buy that number an exemption. We also explored a token bucket of the same rate (burst 4 or 12; its challenge-tier bucket refills independently, unlike RFC 2697's (Heinanen and Guerin, 1999)) and the counter combined with the tests.

3.6. Capacity controls

Step 9 multiplies its caps by 0.25 to 1.5 according to a baseline job comparing each key with its expected rate, learned from the pipeline's hourly counters; without history, the static cap stands. Verified history avoids adaptive reduction, not the static caps or the budget, which bounds SMS count and spend, not vendor or fallback costs.

3.7. Leakage estimates

We compare the SMS sent to a pumper's number over an attack of $T \leq 60$ minutes, by a pumper that solves no challenge and holds no verified history. Against the sequential tests, one reaching B blocks with N requests at rate λ leaks about

$$L \approx \min(N,\ kB + \lambda\tau), \tag{2}$$

before its first verdict, where $k = \lceil h/1.50 \rceil = 5$ and τ, the time a send stays unresolved, is the resolution timeout plus the delivery delay plus half the timeout worker's period (2.55 minutes) if the carrier never verifies, and seconds if it verifies at once. Judged, such a pumper is held for the verdict's hour; therefore, for $T \leq 60$ Equation (2) also estimates the attack's total, ignoring prior credit and outage suspension. A carrier verifying a share σ of codes with a human-like delay gives the conversion statistic an expected increment of $\sigma(-2.08)+(1-\sigma)(1.50)$, zero at $\sigma^* = 0.42$; above it, the statistic drifts away from a verdict.

A counter of q sends per block opens a window with the block's first reserved SMS and closes it after the window length. A pumper that finds its blocks' counters empty (a cold start) sends at most

$$L_c \leq \min(N,\ qBw), \qquad w = \lceil T/\text{window} \rceil, \tag{3}$$

whatever its carrier does with the codes. If a legitimate send has already opened a window on block b, using $c_b \geq 1$ slots, the pumper can also take the remaining $(q - c_b)^+$ before at most w further windows open, so in general

$$\begin{aligned} L_c &\leq \min\Big(N,\ qBw + \textstyle\sum_{b \text{ open}} (q - c_b)^+\Big) \\ &\leq \min(N,\ B\,(qw + q - 1)). \end{aligned} \tag{4}$$

Solving challenges doubles these bounds, and a verified history exempts a client from them. While neither policy's estimate reaches N, the cold-start bound lies below the tests' estimate when

$$B\,(qw - k) < \lambda\tau, \tag{5}$$

and a warm start adds the residual quota to the left side. Equation (5) is the algebraic crossing of two simplified quantities, neither of which is an empirically established switching point or a necessary condition: other limits can hold either policy below its estimate. A carrier that games outcomes may never be judged, so the tests' leakage approaches N.

In the code, every send takes a slot, verified clients' included. If a block's sends arrive as a Poisson stream with mean m per window and all reach the counter, a window holds $1+X$ of them, $X \sim \text{Poisson}(m)$, and the idealized share beyond the quota is $E[(1 + X - q)^+]/(1 + m)$: 7.3% at $m = q/2$ and 27% at $m = q$ for $q = 4$. This is not a challenge rate: exempt clients take slots without challenge, and challenge failures, the second tier and earlier gates are ignored. The estimates use only the rules, configuration, and attack rate; what an excess send costs a user needs the simulation.

4. Methodology

Figure 5 summarizes how we evaluated the design. Our implementation of Algorithm 2 runs inside an event-driven simulator against scripted attackers and simulated users, one run per design, attacker, workload and seed (Algorithm 3); we group the runs into studies under two protocols that we committed before their runs; and we score each run by what the attacker got through and by what legitimate users lost. The two dashed loops of the figure are the method's two disciplines: settings selected on tuning seeds are frozen before they meet held-out seeds and workloads, and the leakage estimates derived from the rules alone (Section 3.7) are checked against the measurements. The rest of this section gives the implementation, the simulator, the protocols and the metrics in that order.

4.1. Implementation

We implemented the pipeline of Algorithm 2 in about 4 000 lines of Python on in-memory and Redis backends. Up to release 2.8.3, the in-memory store that our simulator uses let a sorted set expire a fixed time after its first write, whereas Redis kept it while written to, so the simulated outage detector and Step 5 lost recent observations. Release 2.9.0 follows Redis in every rule we compared, which differential tests check, and we reran every study with it; in earlier releases we repaired seven replay and concurrency defects, each with a regression test. Of our 453 tests, 19 run against a real `redis-server` and 18 exercise specific interleavings of two instances, not all. Step 11 is still not one transaction: a crash after its reservations leaks reserved slots or resolves the send as undelivered. The implementation is a research prototype.

4.2. Simulation

We run the pipeline against scripted vendor fakes on a simulated clock, as Algorithm 3 sets out: every exogenous quantity of a request is drawn before the pipeline sees it, from one random stream for legitimate traffic and another for the attacker, so every scenario at a seed, the attack-free one included, offers the same users, and our comparisons between scenarios are paired. After a warm-up of ten minutes (30 or 130 for pumping), the attack runs over legitimate traffic at 20 requests per minute. Our simulated users verify 80% of codes after a log-normal delay (median 22 s; 2 s for the 20% using autofill); 70% of numbers are reachable on WhatsApp; 90% of users complete a challenge; 20% are returning account holders with a stable fingerprint, a trusted number and verified history, so our results for returning users describe trusted identities only.

Three behavioral parameters have published counterparts: genuine conversion (one provider's 68% or more (Twilio Inc., 2026b); ours 60%–90%) and the never-converting destinations and identity churn of Section 2; the rest are our assumptions. Table 2 groups the attacker profiles we scripted by capability and marks the destination policies each faced (S, sequential tests; C, counters; B, block-limit; parameters in `results/attacker_profiles.md`). Attackers draw pool size, rate and CAPTCHA class per seed, which explains 92% [80, 98] of the variance of the farm's leakage in a nested design (ten configurations, three seeds each). Brackets in the text and tables, and bars and bands in the figures, are 95% percentile-bootstrap intervals over seeds, paired per seed for differences; they describe variation under our generator, not uncertainty about real traffic.

4.3. The two protocols

We wrote two protocols and committed each before its runs; a supplementary map (`results/study_map.md`) gives each study's seeds. The first fixed the matched comparison and a robustness study over 12 Latin-hypercube points (conversion 0.6–0.9 and five other behavioral parameters) and a held-out family (60 to 120 attack requests a minute, 50 000 to 500 000 addresses, heterogeneous legitimate traffic), with claims C1–C5 that we count as holding if true in 90% of cells, a benchmark convention, not a confidence level. The matched comparison changes only the destination policy and, at three block densities, selects on tuning seeds the least leakage summed over four concentrated pumpers among settings at or above the *tuning target*, the default's completion minus 0.5 points. The robustness study's first run failed C1 in 32 of 108 cells because attack and no-attack runs drew different users; we fixed the pairing, a fix that followed a result, and reran every study. We wrote the second protocol (claims K1–K5) after the matched comparison had selected three counters and committed it before its runs; six of its seed values had served other robustness points, never

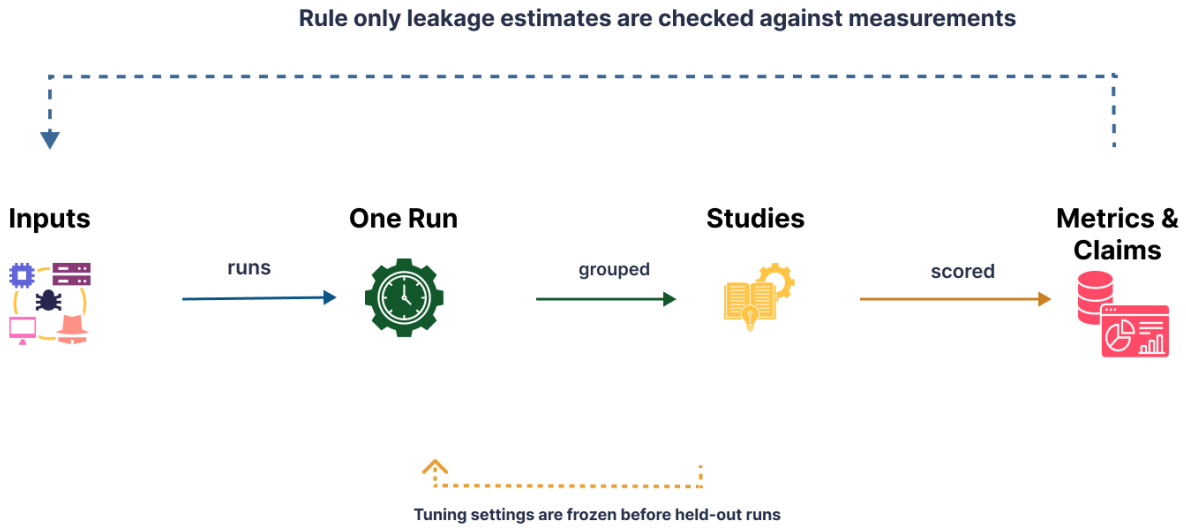


Fig. 5. How we evaluated the design.

Algorithm 3 One simulation run, as the harness executes it.

Input: a design (its feature set and settings), an attacker profile a, a legitimate-traffic specification ℓ, a seed, and T attack minutes

derive three random streams from the seed: rng_ℓ (users), rng_w (attacker) and rng_p (whether a challenged person solves the challenge)
▷ *the same seed offers the same users to every scenario, so comparisons are paired; a digest of the offered trace is checked across designs*
build the pipeline of Algorithm 2 on the in-memory store, with fakes for CAPTCHA, address intelligence, HLR, the sender and WhatsApp; seed the returning population (20%) with verified history
for minute = −warm-up, . . . , T − 1 **do** ▷ warm-up 10 min, or 30 or 130 for pumping; the attack starts at minute 0
    $n_\ell \sim$ Poisson(ℓ.rate) from rng_ℓ and $n_a \sim$ Poisson(a.rate) from rng_w; draw each arrival's time inside the minute and sort
    for each arrival at time t **do**
        apply every event due by t (receipt, code entry, resend) at its own time, then set the clock to t
        draw the request's exogenous quantities in advance: identity, address, number, CAPTCHA scores, delivery delay, whether and when the person would verify, WhatsApp reachability
        $r \leftarrow$ ProcessRequest(request) ▷ Algorithm 2; a challenged user retries with probability 0.9, a challenge-buying attacker always
        if r sent an SMS **then** schedule the receipt at t plus the queue and delivery delays, and a code entry after the recipient's delay if it verifies
    ▷ *a pumper's carrier verifies as its profile says: always within 1 s, 60% after 30 s, or only when the block's statistic nears its threshold; a receipt faker schedules a failed receipt*
    apply events to the minute boundary; run the feedback loop's resolution timeouts (OnOutcome of Algorithm 2); run the adaptive job of Step 9 at its cadence
    ▷ *oracle job: told the legitimate rate; learned job: reads the pipeline's own counters, so cold start and poisoning are in play*
    record the attack requests offered and the SMS leaked this minute, and every user's progress through the funnel
drain: apply events until the receipt grace, the resolution timeout, the slowest code entry and a resend have elapsed after the last minute
return leakage per minute and containment; each user's funnel (dispatched, delivered, completed, challenged, refused) by cohort; verdict events; the attacker's verifications, challenges solved and bill; the configuration, seed, workload digest and code hash

Table 2
The 25 attacker profiles, and the destination policies each one faced.

Capability	Profiles	Policies
A signature in the request (5)	One address; datacenter rotation abroad; a premium-rate prefix; numbers walked upward; the farm with an unattested iOS header	S, B
Looks like a person (7)	Residential bot; farmed CAPTCHA with fresh or aged fingerprints; one 48-hour-old browser profile	S, B
	Low and slow, 2 or 10 requests/min over 20 ASNs; datacenter rotation buying challenge solutions	S
Games outcomes or trust (8)	Pumpers on elevated-cost or standard ranges verifying within 1 s, or 60% after 30 s	S
	Threshold-aware and receipt-faking carriers; three trust builders (500 identity/number pairs verify for 10 or 30 min, then flood)	S, C
Concentrated on the destination (5)	Four pumpers on three blocks (carrier never verifies, verifies within 1 s or 60% after 30 s, or buys challenges); a poisoner on users' blocks	S, C
Spread (generated)	3 to 300 blocks, at full rate or at most the counter's quota per block	S, C

selection. We reran both unchanged with 2.9.0 (9 919, 7 925 and 240 runs, each carrying its code hash); against 2.8.0, total leakage changed in 855 of 17 964 comparable runs, but the principal selections, eligibility decisions and claim counts did not; one auxiliary setting (false alarms matched at unbounded credit) moved from T1000 to T300 under both protocols (artifact change report).

Table 3
What each design leaks, and what users pay for it.

	caps lifted: leaked			caps on, v2		
Attacker	v1	BL	v2	leaked	refused	completed
Single client	5.6	0.3	0.3	0.3	1.1	79.1
Datacenter rotation	69.0	69.0	0.0	0.0	1.1	79.1
Premium pumping	99.4	1.0	0.0	0.0	1.1	79.1
Sequential numbers	99.4	1.0	11.5	10.7	3.5	77.5
Reused profile	99.5	99.5	22.5	17.4	8.6	74.2
Residential bot	37.2	37.2	37.2	29.9	16.2	69.6
Farmed CAPTCHA	99.5	99.5	99.5	45.6	54.0	45.5
Aged fingerprints	99.5	99.5	99.5	45.6	54.0	45.5
Forged header	100.0	99.5	99.5	45.6	54.0	45.5

4.4. Designs and metrics

We evaluate seven designs: **v1**, **v1-corrected** (attested platform), **budget-only** (sessions, static caps, budget), **block-limit** (a refusing counter of five sends per block per day), **conversion-only** and **speed-only** (v2 minus one block test), and **v2**. We call a seed *contained* when leakage stays at most 5% of the attack rate from some minute to the window's end, for five minutes or more. *Attributable* loss compares the same requests with the policy enforced and absent, net of the reverse, in points of offered users: on attack-free traffic we call it the *benign cost*, on the attacked trace the *extra harm* relative to the attacked no-policy arm. *Degradation* compares the attacked trace with attack-free operation under the same policy.

5. Results

5.1. What is contained

Table 3 gives the leaked share (%, mean of per-run shares) of about 600 attack requests in 20 minutes over 30 seeds under v1, block-limit (BL) and v2 with source caps lifted, and under v2 with caps on, with the first-time users refused and all users completing (%); supplementary Table S1 (`results/design_table.md`) gives every design with intervals. Datacenter rotation leaks nothing under v2 because it scores in the challenge tier and declines challenges; Step 5 refuses the premium range, and the session cap holds the single client to two messages (Table 3); in all 108 robustness cells, these attacks leaked under 5% at completion within two points of the paired no-attack run (claim C1). The walk leaks 61 [53, 70] messages before its block's verdict (block-limit: five), and the reused profile 89 [83, 95].

The farm leaks 99.5% with caps lifted under every design; v2's adaptive cap holds it to 229 [215, 244] messages, 46%, by refusing 54% [47, 60] of first-time users, so 45.5% complete instead of 76.4% under v1: rationing (claim C5 held in 36 of 36 cells).

5.2. What each layer does

In the ablation grid (90 effects, each conditional on all other layers, unadjusted intervals; artifact), removing the adaptive caps lets the farm leak 310 [194, 421] more messages; the risk engine, feedback loop, and block key hold the walk by 166, 152, and 121 messages, and the loop holds the reused profile by 138. The key acts only through the loop (interaction −121 [−141, −101]).

5.3. Where reputation stops separating

A farmed-CAPTCHA attacker uses an address and fingerprint once and shares its country, prefix, and residential ASN with every legitimate user; thus, its failures penalize everyone: over an hour with caps lifted (five seeds), up to twice the legitimate rate, 99 to 100% of it leaks, since a shared key stays above 0.3 conversion until failures outnumber legitimate sends about 1.7 to one. At ten times, 70% leaks while 17% of legitimate users are challenged and 3.2% refused (claim C2, at least 90% leaked with caps lifted, held in 34 of 36 cells). No single score change separates better: a fresh-fingerprint weight of 40 stops it only by challenging 47% of legitimate users. In the settings run, reputation separated an attacker only on keys its sends dominated (Douceur, 2002).

5.4. What rationing costs

When separation fails, the cap is the lever: over ten seeds, the default lets 47% of the farm's requests through and refuses 41% of all legitimate users, the tightest setting 20% and 64%. Both assume an oracle job that is told the legitimate rate and runs every minute; running every five minutes or hourly, it lets 75%–96% through. Learning the rate from the pipeline's counters, the deployed job leaks 193 [161, 229] messages with a clean three-week profile, 340 [239, 441] more on a cold start, 106 [62, 151] more with a stale profile, and 45 [10, 77] more with a poisoned one.

5.5. Pumping against the tests

Table 4 covers concentrated pumpers on three blocks with caps lifted over ten seeds: messages leaked of about 574 requests in 20 minutes, seeds contained, mean minutes to containment (contained seeds) and to the first verdict (seeds with one), and codes entered. The speed test catches a carrier verifying within a second at 19 messages, while the carrier verifying 60% with human-like delay is contained in none of the ten 20-minute runs: its first verdict comes at minute 11.6 on average and 452 messages leak; in an hour, 1 067 leak and two of ten runs are contained, at minute 47.0 on average. Claim C3 (containment with at most 30% leaked) held in 36 of 36 cells.

Across 18 spread configurations, leakage follows the blocks touched: with 30 blocks, five to seven runs in ten are still contained. Unsaturated, Equation (2) errs by 1.6 to 7.9 messages per run (1%–13%, ten configurations); saturated, it overstates by up to 91.

5.6. Counting against testing

5.6.1. At a service target

Table 5 reports the first protocol at 200 blocks (144 legitimate sends per block a day; tuning seeds 100–102, evaluation seeds 0–9): 20-minute leakage against the never-verifying (NV), instant (IV), human-like (HL) and challenge-buying (CB) carriers

Table 4
Concentrated pumping: how much leaks before the tests contain it.

Carrier, design	Leaked (msgs)	Contained (seeds)	Containment (min)	First verdict (min)	Codes entered
never verifies: v1	570 [419, 736]	0/10	–	–	0
v2 default	90 [71, 114]	10/10	3.5 [3.1, 3.9]	3.0	0
verifies within 1 s: v2 default	19 [17, 21]	10/10	1.4 [1.1, 1.7]	0.4	19
verifies 60% after 30 s: v2 default	452 [365, 539]	0/10	–	11.6	272
v2 default, 60 min	1 067 [839, 1 310]	2/10	47.0 [42.0, 52.0]	25.5	–
buys challenges: v2 default	106 [87, 127]	10/10	4.3 [3.8, 4.9]	3.0	0

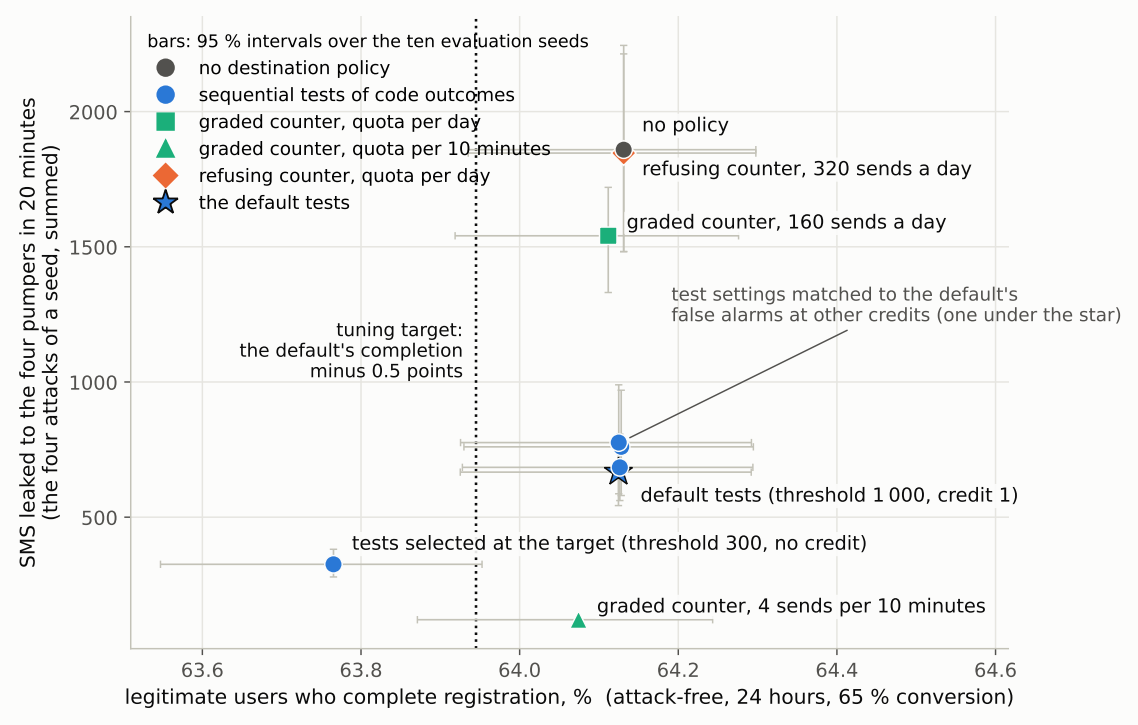


Fig. 6. Each setting's leakage against the completion it costs.

Table 5
Counting against testing at the service target.

Setting (selected as)	NV	IV	HL	CB	Compl.	Benign	Chall.	Events
none	319	566	570	405	64.13%	–	1.53%	0
sequential $T1000$, $c1$ (default)	90	19	452	106	64.12%	0.01	1.64%	6.5
sequential $T300$, $c0$ (selected at tuning target)	88	14	126	98	63.77%	0.37	6.54%	334.3
counter, graded, 4 per 10 min (selected)	24	24	24	48	64.07%	0.06	2.38%	0
counter, graded, 160 per day (selected daily)	319	409	408	405	64.11%	0.02	1.87%	0

and, from 24-hour attack-free runs at 65% conversion, completion, benign cost (points), users challenged and verdict events a day; the tuning target is 63.95%. Figure 6 plots each setting's completion against its leakage summed over the four pumpers' runs of a seed (not one combined attack), with the tuning target dotted. The figures name each setting by its rule, threshold and credit, so $T300$, $c0$ of Table 5 is the test with threshold 300 and no credit; the three points beside the default, one under its star, are sequential settings matched to the default's false alarms at other credits (key: `results/figure1_key.md`). Figure 7 gives, carrier by carrier, each selected setting's leakage minus the default's, paired per seed (left of zero: leaks less than the default). The sequential setting selected at the tuning target holds the human-like carrier to 126 messages only by raising 334 false verdict events a day, and misses the target by 0.18 points on seeds 0–9. The graded counter of four sends per block per ten minutes leaks 24 against each outcome-reading carrier and 48 against the challenge buyer, at 0.06 points; counters selected at the sparser settings leak 6 to 18 at 0.15 to 0.35 points (up to 0.43 at 80%). With the full grid and density-specific attacks (E5) the rule selects the same counters, and so do other objectives on the tuning runs (worst-case leakage, a first-time-loss constraint, each attacker alone, 2 000 random mixes), except the instant verifier alone and 3 mixes at 200 blocks; on

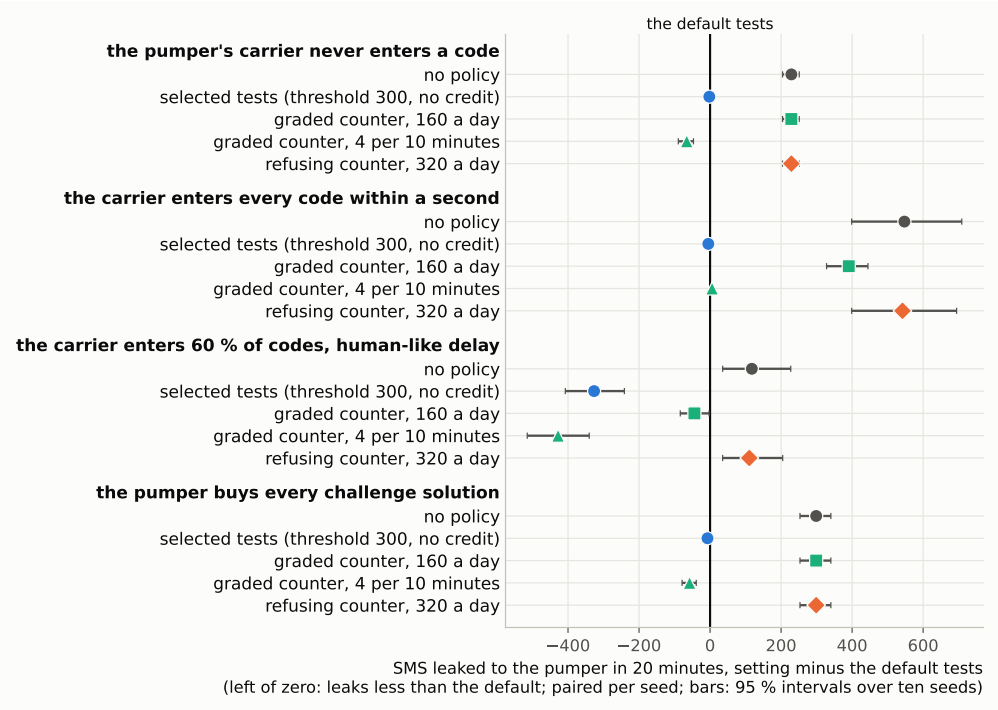


Fig. 7. Each selected setting against the default tests, carrier by carrier.

Table 6
The frozen counter on seeds and workloads it had never seen.

Claim	Statement	True / cells	IV	Result
K1	Leaks at most 10% of attack requests	144/144 (100%)	36/36	holds
K2	Attack-free attributable loss at most 0.5 points	36/36 (100%)	–	holds
K3	Attributable loss under attack at most 0.5 points	127/144 (88%)	32/36	fails
K4	Leaks no more than the default sequential tests	131/144 (91%)	23/36	holds pooled, not for IV
K5	Attributable loss under attack at most the default's plus 0.25 points	93/144 (65%)	23/36	fails

fresh seeds the counter leaked least of the winners against every attacker but the instant verifier (25, against 15 and 16). On seeds 300–309 the 1 000-block counter falls 0.12 points below its target, yet no seed-and-conversion cell fails the paired check (the evaluation default minus 0.5 points).

5.6.2. On held-out traffic

Table 6 covers 12 shifted points of the held-out family, with held-out attack rates and pools, three seeds each and a 20-minute pumper on three blocks real users share; a claim holds if true in 90% of cells, which share workloads and seeds and are not independent, and IV marks the instant verifier's cells. Frozen and run on this family, the counter leaked 33, 34, 33, and 62 messages against the four carriers, where the default leaked 287, 36, 344, and 287, and no policy 451 to 1 857, at a benign cost of 0.13 [0.09, 0.17] points, and failed both service-under-attack claims: 13 of the 17 cells that broke K3 lie at the three points with the most legitimate traffic, where users and pumper compete for one quota; K5 failed in 51 cells over 11 of 12 points. The counters selected for sparser traffic cost 4.7 and 5.8 points here even without an attack: the quota must match the traffic.

Tables 7 and 8 report E1 at 200 blocks (60-minute attacks, seeds 300–309): a pumper on three blocks real users share, and a poisoner flooding all 200 blocks. Leakage under the pumper is summed over the four carriers' runs of a seed, which can hide a carrier on which a policy loses (per carrier: artifact); harm is net completions lost in points, averaged over the carriers (negative: a gain), with intervals for leakage and the attacked blocks in Table 7 and for all users' extra harm in Table 8. In Table 7, against the attacked no-policy arm, the counter cost all users nothing on average and gained the attacked blocks' users three points on average, since without a destination policy, the pumper exhausts their blocks and the hourly budget; with the human-like carrier alone, it cost them 1.9. Against the attack-free operation, every evaluated policy left those users 5 to 9 points worse off on average (2 to 12 per carrier; 8 with none): none shields a block an attacker shares. Counter and tests together, explored outside the frozen claims, leaked least summed over the four carriers but not against the instant verifier (28; $T300$, $c0$: 22), and under the poisoner (Table 8) cost all users most of the counters and buckets and leaked more than $T300$, $c0$. A token bucket with a burst of 4 spared the poisoner's users only by letting almost everything through; with two untuned settings, neither limiter

Table 7
A pumper on blocks that real users share: what leaks and who pays.

Policy	Leaked (SMS)	Extra harm against attacked, no policy: all users (≈1 200)	Extra harm against attacked, no policy: attacked blocks (≈18)	Degradation against attack-free, same policy: all users	Degradation against attack-free, same policy: attacked blocks	Benign cost: all users
none	5 509 [4 409, 6 567]	–	–	0.19	8.0 [5.1, 10.7]	–
sequential, default	1 149 [953, 1 340]	−0.07	−1.0 [−2.8, 0.8]	0.10	7.0 [4.7, 9.1]	0.02
sequential $T300$, $c0$	397 [312, 481]	0.13	0.9 [−1.3, 3.3]	0.12	8.9 [6.0, 11.7]	0.20
counter, 4 per 10 min	403 [386, 418]	−0.05	−3.2 [−5.0, −1.1]	0.07	4.8 [2.9, 6.7]	0.07
counter + sequential	190 [179, 201]	−0.04	−3.1 [−5.1, −0.7]	0.07	4.9 [3.0, 6.8]	0.08
token bucket, burst 4	424 [412, 437]	−0.12	−3.1 [−5.1, −0.7]	0.07	4.9 [2.8, 7.0]	0.00
token bucket, burst 12	545 [531, 559]	−0.11	−2.1 [−4.3, 0.2]	0.08	5.9 [3.3, 8.8]	0.00

Table 8
A poisoner on every block: what users lose.

Policy	Leaked (SMS)	Extra harm against attacked, no policy: all users	Extra harm against attacked, no policy: first-time users
none	1 976	–	–
sequential, default	1 357	1.09 [0.59, 1.59]	1.36
sequential $T300$, $c0$	858	2.55 [2.04, 3.07]	3.20
counter, 4 per 10 min	1 662	0.72 [0.44, 1.02]	0.91
counter + sequential	1 286	1.43 [0.88, 1.97]	1.79
token bucket, burst 4	1 877	0.12 [0.04, 0.23]	0.16
token bucket, burst 12	1 974	0.00 [0.00, 0.00]	0.00

dominated, and we ranked only the implemented policies. Returning users, all simulated with verified history, lost nothing.

5.6.3. The counter's boundary

Figures 8–10 show the counter's operating boundary (E4, seeds 300–304). Figure 8 plots the attributable loss of the three hot blocks' users against their legitimate rate, attack-free and with a never-verifying pumper on those blocks; its horizontal axis is logarithmic, and lines join sampled settings without locating a crossover. Attack-free, the counter costs a block's users nothing at one send per ten minutes, 2.3 points at four and 5.5 at eight, far below the idealized share beyond the quota (27% at four), since most challenged users complete or switch to WhatsApp; without WhatsApp, eight sends cost 13.2. A 30-minute launch at three times the normal rate costs its blocks' users 16 points. With a pumper on the same blocks, both policies turn that cost into a gain at up to four sends per ten minutes (the counter 6 to 15 points, the tests 1 to 10), because without a policy the pumper exhausts those blocks and the hourly budget; at eight sends, both cost their users again, the counter 4 points and the tests 9.

Figures 9 and 10 plot leakage in 20 and 60 minutes against the blocks a pumper uses, at full rate or paced to the quota, with Equations (2)–(4) for the full-rate pumper as hollow markers; both axes are logarithmic. In twenty minutes (Figure 9) the

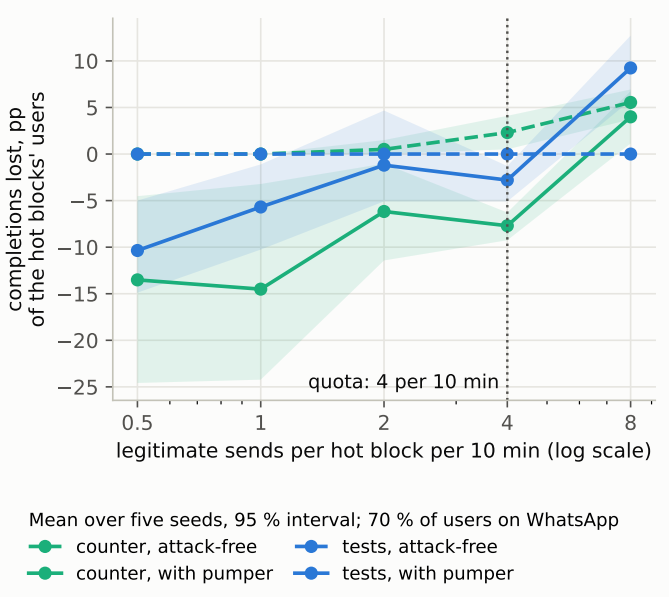


Fig. 8. What the counter costs the users of a busy block.

counter leaked 25 and 81 messages on 3 and 10 blocks where the tests leaked 110 and 144, the two leaked about the same on 30 blocks (235 against 242), and from 100 blocks on both approached the no-policy line of about 700. Over 60 minutes (Figure 10) the default leaked 110, 144 and 242 messages on 3, 10 and 30 blocks, against 105, 139 and 240 from Equation (2) ($\lambda\tau \approx 90$), and the counter 71, 240 and 679 against its cold-start bound of 72, 240 and 720. Once 7 messages sent, exempt, to real users' trusted numbers are set aside, no counter run that started cold (44 of 100) exceeded Equation (3); three warm ones did, by 5 or 6 messages, and none exceeded Equation (4). With these unfitted inputs Equation (5) ordered the policies' mean leakage in 18 of 18 decisive cells (two ties at N), and with the warm-start term in 17, missing where the estimates differ by 0.8 messages (30 blocks, 20 minutes; measured

difference 6.6 [−27.2, 42.4]); per seed, 76 of 78 decisive comparisons agree. The algebraic crossing lies near five blocks for an hour and 30 for twenty minutes, where the nearest measured differences (three and 30 blocks) do not exclude zero; it is not a measured switching point. Paced to the quota, a pumper leaks about as much under the counter as with no policy (71 against 72 on three blocks in an hour), and 23 under the tests.

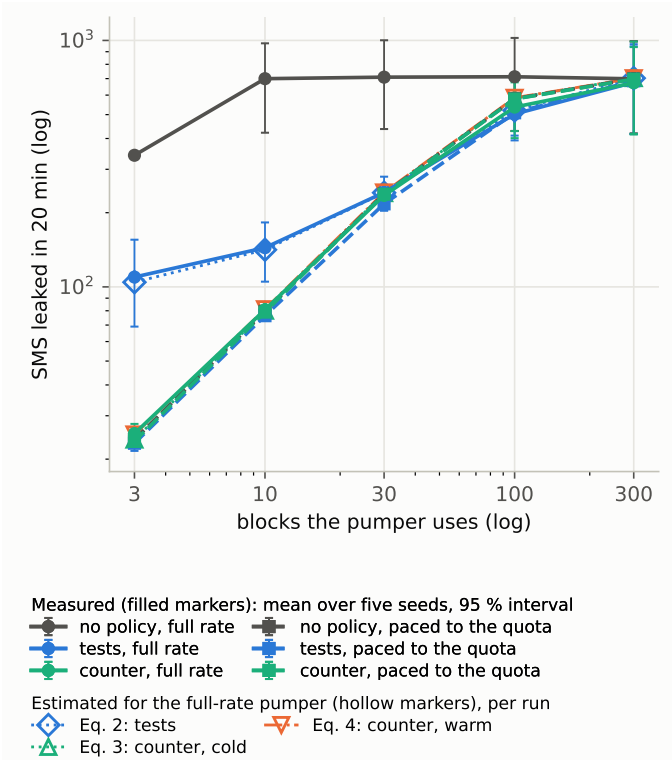


Fig. 9. Leakage against the blocks a pumper spreads over, in twenty minutes.

Against adaptive attackers (E3), the counter held the threshold-aware carrier, receipt faker, and concentrated trust builder to 24 to 36 messages (502 to 665 under the default), but not trust builders on random numbers (577, as under the default); over 360 minutes, the human-like carrier gained 81 messages an hour under the counter, and 753 under the default.

5.7. The cost to legitimate users

On one block, 20 000 Monte Carlo trials per cell give the probability of a verdict within 30 legitimate sends: 0.09% [0.06, 0.14] at 80% conversion with 20% autofill, 1.9% at 65% and 19.5% at 50%. Over 24 hours, the default raises 0.5 verdict events a day on 200 busy blocks at 80% conversion and 6.5 at 65%, where they meet 30 of about 28 800 requests (claim C4 held in 18 of 18 cells).

A poisoner flooding for 20 min the 20 blocks legitimate users concentrate on (caps lifted, ten seeds) raises 16 verdict events meeting 98 legitimate requests, 33 never completing; against observe-only operation on the same trace, enforcement costs 12.7 [4.6, 22.9] completions, 3.1% of users (6.3% without WhatsApp; 23.9% with a 24-hour denylist). One that stops

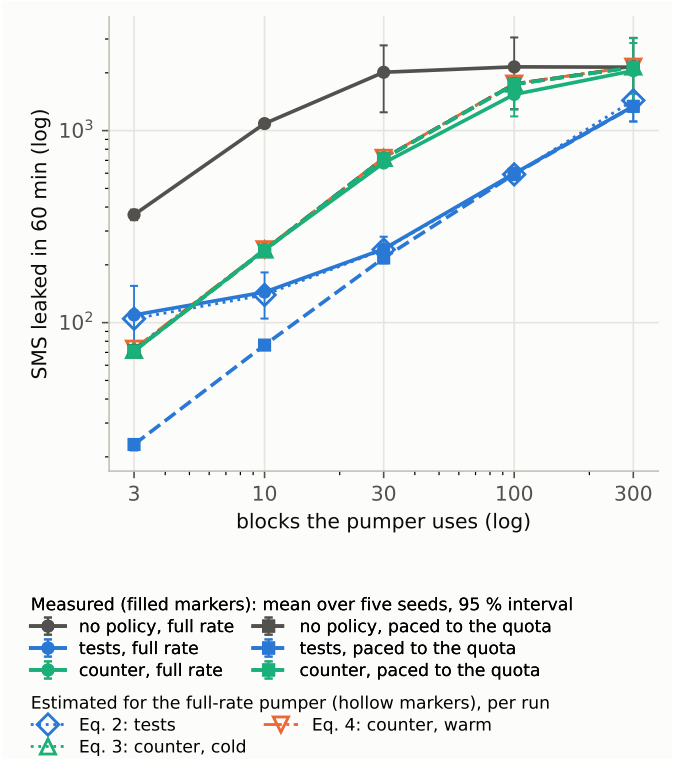


Fig. 10. The same, over an hour.

after ten minutes leaves verdicts meeting 406.5 requests afterwards, an attributable loss of 46.2 [16.9, 79.8]. Grading limits the harm; it caps it neither at the challenge nor at the attack's duration.

5.8. Adaptation, economics and overhead

The leakages of these scripted adaptations are not bounds. Against v2 with caps lifted, a datacenter attacker buying every challenge solution leaks 55, and a threshold-aware carrier and a receipt faker leak 570 each, drawing no verdict.

If pumpers earn a share of the 0.1422 USD retail price and pay an event-level bill, they break even at a share of 0.043 under v1 and 0.054 to 0.271 under v2 (the instant verifier 1.310); over 360 minutes under the counter, the human-like carrier needs 1.048. These are break-even shares for scripted attackers under assumed prices, not bounds or criminal profit.

On a real Redis (four vCPUs, four workers), we measured sends at 13.8 ms (median) and 21.1 ms (99th percentile), and 133 requests per second passed at concurrency 128 with the 400 ms floor and 50 ms vendor calls. With fast vendors, response times distinguished no pair of five outcome classes (smallest Kolmogorov–Smirnov $p = 0.051$). Under an adversarial mixture, 81% of sends overran the floor, and response-time thresholds separated sends from refusals with 88% balanced accuracy on the fitted run and, fitted on a new run, 95 to 99.9% on two others: a separation on that machine, not a deployment accuracy.

Table 9
Every design against every source-side attacker, caps lifted.

Attacker	v1	v1-c.	budget	BL	conv.	speed	v2
Single client	5.6	5.6	0.3	0.3	0.3	0.3	0.3
Datacenter rotation	69.0	69.0	69.0	69.0	0.0	0.0	0.0
Premium pumping	99.4	99.4	94.9	1.0	0.0	0.0	0.0
Sequential numbers	99.4	99.4	99.4	1.0	11.5	13.3	11.5
Reused profile	99.5	99.5	99.5	99.5	22.5	22.5	22.5
Residential bot	37.2	37.2	37.2	37.2	37.2	37.2	37.2
Farmed CAPTCHA	99.5	99.5	99.5	99.5	99.5	99.5	99.5
Aged fingerprints	99.5	99.5	99.5	99.5	99.5	99.5	99.5
Forged header	100.0	99.5	99.5	99.5	99.5	99.5	99.5

6. Discussion and limitations

6.1. Lessons

In the scripted workloads, carriers that verify with human-like delay, fake receipts, or know the thresholds defeated our outcome tests but not a short-window counter. Within an hour of a verdict, the counter's allowance, unlike the tests', grows with the windows an attack spans; over longer campaigns, verdicts expire and the tests leak again. Equation (5) summarizes this for pumpers that solve no challenge and hold no exemption. Its inputs are in an operator's logs; the cost of an excess send, competition under attack, and exemption abuse come only from the simulation. The counter and tests together, with a quota set from each block's traffic, is a hypothesis for real traffic, not a recommendation.

6.2. Against the benchmarks

Table 9 sets the seven designs of Section 4 against the nine source-side attackers on the same traffic and seeds, with source caps lifted so that only the behavioral layers act (intervals in supplementary Table S1). Each design adds one thing to the baseline, and the table says what it bought. Correcting the platform header alone (v1-corrected) changed only the forged-header attack, from 100% to 99.5%: that attacker is the farm with a header, and the header was never what held it. Sessions and a hard budget (budget-only) removed the single client's leakage (5.6% to 0.3%) and cut the premium pumper's from 99.4% to 94.9%, nothing else. A refusing counter of five sends per block a day (block-limit) held the premium pumper and the walked numbers to 1%, the two attacks that concentrate on a few blocks, and left the rest where v1 had them. The behavioral layers of v2, with either block test, were what removed datacenter rotation (69% to 0) and most of the reused profile (99.5% to 22.5%); the conversion and speed tests differ only on the walked numbers (11.5% against 13.3%). No design in the table separates the residential bot (37.2%) or the farm (99.5%): those are held only by the adaptive caps of Table 3, at the price Section 5.4 reports. On the destination, the in-study benchmarks are the no-policy and default arms of Tables 4, 5 and 7: against the never-verifying pumper v1 leaked 570 messages, the default tests 90 and the selected counter 24, and under the shared-block pumper of Table 7 the counter leaked 403 messages where the default tests leaked 1 149 and no policy 5 509, neither policy costing all users anything on average.

The nearest external work is set out in Table 1, and it can be compared only on what each needs and reports, since none ran on the same traffic. Huh et al. (2025) report 89.6% recall at 0.19% false positives for a supervised model on one service's real logs. The class of attack their model targets, a few requests per identity over many fresh identities, is the farm here, which the behavioral layers do not separate and the caps only ration, at 54% of first-time users refused; what the layers do contain without labels are the attacks with a signature in the request (0 to 11.5% leaked with caps lifted; the farm behind a forged header is the farm) and most of the reused profile. Zaliskyi and Odarchenko (2026) report that fixed-window prefix counts blocked 95% and 63% of two incidents at the price of 58% of a legitimate campaign. The graded counter here is the same family of control, a ten-minute window on a 10 000-number block with a challenge rather than a 24-hour quarantine; it leaked 24 of about 574 requests from each outcome-reading pumper at an attack-free cost of 0.06 points of completion, on simulated traffic with no campaigns in it, and it failed its service claims on blocks the attacker shares. Twilio's products report claimed false-positive rates and no leakage. What we report that none of the three does is an attack-free benign cost, a cost under attack for the users who share a block with the attacker, a selection fixed before the evaluation runs and tested on held-out seeds and workloads, and an implementation anyone can run; what the three have and we lack is real traffic.

6.3. Limitations

We simulated everything and assumed the calibration. We wrote both protocols for our own generator, the second knowing which counters had won. Some results follow from the generator (a perfect premium list, a datacenter attacker that declines challenges, returning users who always present verified history), and our selection rule used an unweighted sum of four scripted pumpers' leakage and a mean service target met on tuning seeds, which guarantees no service level elsewhere. Close rankings rest on ten or fewer seeds, and costs that round to zero are not shown equivalent. Per-client quotas within a block, carrier spend ceilings, bounded exemptions, tuned token buckets, and other sequential statistics lie outside the evaluated policies. Vendor adapters are untested live, and the timing result holds only for the observer tested. Destination policies challenge people for what strangers sharing their block did, and deployment under the GDPR or Saudi Arabia's PDPL needs a lawful basis and a route for refused users.

7. Conclusions and future work

We asked how a service that verifies phone numbers by SMS should defend an endpoint that must answer strangers, and in particular which of two label-free destination signals, the number of sends to a number block or the outcomes of the codes sent, it should act on when a pumper is paid on the destination. We answered it in simulation: we ran our twelve-step redesign of a seven-control baseline against 25 scripted attacker profiles

and six other designs, and compared the two destination policies under two protocols we committed before their runs, with the cost to legitimate users made part of the result.

On the source side, our design contains the attacks that carry a signature in the request, a single address, a datacenter range or a premium prefix, at a low cost to simulated users, holds walked numbers and a reused profile to 11.5% and 22.5% of their requests, and the ablation attributes these gains to the adaptive caps, the risk engine, the feedback loop and the block key. Against a residential attacker that resembles a person, outcome-based reputation stops separating once its failures land on keys that legitimate users share; in the settings run, it separated an attacker only on keys its sends dominated. What remains is rationing, whose cost depends on how well the cap tracks the legitimate rate: the deployed job leaked 193 messages with a clean three-week profile and 340 more on a cold start, while the default cap refused 41% of legitimate users. On the destination, counting sends leaked less in aggregate than our outcome tests against concentrated pumpers, although not reliably against an instantly verifying carrier; the frozen counter failed its service claims when the attacker shared blocks with real users and lost its advantage against wide, paced and trust-building pumpers. Within an hour, unfitted estimates ordered the policies' mean leakage in 18 of 18 decisive boundary cells with the cold-start bound and 17 with the warm-start bound, missing a near tie; the crossing was algebraic, not a measured switching point. Without an attack, the default tests raise 0.5 verdict events a day on 200 busy blocks at 80% conversion and 6.5 at 65%, and a counter of four sends per ten minutes costs a block's users 2.3 points.

Against the six benchmark designs run on the same traffic (Section 6.2), the behavioral layers are what removed datacenter rotation and most of the reused profile, a per-block counter alone held the premium and walked-number attacks, and no design separated the residential farm. For an operator, the practical content is an ordering, not a recommendation. Against short, concentrated pumping on a few blocks, a counter of sends per block whose quota matches the block's legitimate traffic leaked less than tests of code outcomes and cost its users less, and the inputs of the estimate that orders the two, the attack rate, the blocks touched and the time a send stays unresolved, are in an operator's logs; the cost of an excess send, the competition for a quota on a shared block and the abuse of exemptions come only from the simulation. The quota must match the traffic: counters selected for sparser blocks cost 4.7 and 5.8 points on busier ones even without an attack. Every number describes scripted attackers and simulated users under assumed calibration, not real traffic, and close rankings rest on ten or fewer seeds.

Replaying real traffic is the experiment our work most needs: it would test the assumed calibration, the cost of an excess send to users who share a block, and the quota a counter needs per block. The policies left outside the evaluation, per-client quotas within a block, bounded exemptions, tuned token buckets and windowed or time-uniform statistics, could then be compared under the same predeclared protocols.

Declaration of generative AI and AI-assisted technologies in the writing process

The author used Claude (Anthropic) for code, evaluation, drafting and editing; checked numbers against the recorded results and claims against the code; and takes full responsibility for the content.

CRediT authorship contribution statement

Naveed Ahmed: Conceptualization, Methodology, Software, Validation, Investigation, Writing – original draft, Writing – review & editing.

Declaration of competing interest

The author declares no known competing financial interests or personal relationships that could have appeared to influence this work.

Data availability

No human or real traffic data was used. Code, protocols (`config/`), every run's configuration and outcome and the sources of headline numbers (`results/`) are at https://github.com/nahmedpsu/OTP-DDOS, release 2.9.1 (tag v2.9.1), which runs unchanged the code of release 2.9.0 (tag v2.9.0, commit 9af3100) that produced the results (`code_identity.md`); the 2.8.0 results are in `results/historical_2.8`. These files are the supplementary material.

References

Ahmed, N., 2026. OTP Guard: a layered SMS validation, risk-scoring and rate-limiting pipeline against OTP flood abuse (version 2.9.1). https://github.com/nahmedpsu/OTP-DDOS, tag v2.9.1. Results produced by 2.9.0 (commit 9af3100), whose executable code 2.9.1 keeps; 2.8.0 results (commit 1c277a0) in results/historical_2.8.

Commsrisk, 2023. Elon Musk says Twitter lost $60mn a year because 390 telcos used bot accounts to pump A2P SMS. https://commsrisk.com/elon-musk-says-twitter-lost-60mn-a-year-because-390-telcos-used-bot-accounts-to-pump-a2p-sms/. Trade press; the figure is the company's own claim.

Dean, D., Stubblefield, A., 2001. Using client puzzles to protect TLS, in: 10th USENIX Security Symposium, USENIX Association.

Dionysiou, A., Athanasopoulos, E., 2020. SoK: Machine vs. machine – A systematic classification of automated machine learning-based CAPTCHA solvers. Computers & Security 97, 101947. https://doi.org/10.1016/j.cose.2020.101947.

Douceur, J.R., 2002. The Sybil attack, in: Peer-to-Peer Systems (IPTPS 2002), Springer. pp. 251–260. https://doi.org/10.1007/3-540-45748-8_24.

Google LLC, 2026. SMS retriever API overview. https://developers.google.com/identity/sms-retriever/overview. Accessed 2026-10-01.

Heinanen, J., Guerin, R., 1999. A single rate three color marker. RFC 2697, Internet Engineering Task Force. https://doi.org/10.17487/RFC2697.

Howard, S.R., Ramdas, A., McAuliffe, J., Sekhon, J., 2021. Time-uniform, nonparametric, nonasymptotic confidence sequences. The Annals of Statistics 49, 1055–1080. https://doi.org/10.1214/20-AOS1991.

Huh, J.H., Shin, H., Ahn, S., Yi, H., Cho, J., Kim, T., Lim, M., Choi, N., 2025. Preventing artificially inflated SMS attacks through large-scale traffic inspection, in: 34th USENIX Security Symposium, USENIX Association, Seattle, WA.

Mi, X., Feng, X., Liao, X., Liu, B., Wang, X., Qian, F., Li, Z., Alrizah, S., Chen, L., Qian, Y., 2019. Resident evil: Understanding residential IP proxy as a dark service, in: 2019 IEEE Symposium on Security and Privacy (S&P), IEEE. https://doi.org/10.1109/SP.2019.00011.

MITRE Corporation, 2026. Resource hijacking: SMS pumping, technique T1496.003, MITRE ATT&CK. https://attack.mitre.org/techniques/T1496/003/. Accessed 2026-10-01.

Motoyama, M., Levchenko, K., Kanich, C., McCoy, D., Voelker, G.M., Savage, S., 2010. Re: CAPTCHAs – understanding CAPTCHA-solving services in an economic context, in: 19th USENIX Security Symposium, USENIX Association.

Mulliner, C., Borgaonkar, R., Stewin, P., Seifert, J.P., 2013. SMS-based one-time passwords: Attacks and defense, in: Detection of Intrusions and Malware, and Vulnerability Assessment (DIMVA 2013), Springer. pp. 150–159. https://doi.org/10.1007/978-3-642-39235-1_9.

NIST/SEMATECH, 2026. CUSUM control charts, section 6.3.2.3, e-handbook of statistical methods. https://www.itl.nist.gov/div898/handbook/pmc/section3/pmc323.htm. Accessed 2026-10-02.

Page, E.S., 1954. Continuous inspection schemes. Biometrika 41, 100–115. https://doi.org/10.2307/2333009.

Richter, P., Wolsing, F., Chandrasekaran, A., Padmanabhan, R., Allman, M., Bush, R., Feldmann, A., 2016. A multi-perspective analysis of carrier-grade NAT deployment, in: Proceedings of the 2016 ACM Internet Measurement Conference (IMC), ACM. https://doi.org/10.1145/2987443.2987474.

Sahin, M., Francillon, A., 2021. Understanding and detecting international revenue share fraud, in: Network and Distributed System Security Symposium (NDSS), Internet Society. https://doi.org/10.14722/ndss.2021.24051.

Sahin, M., Francillon, A., Gupta, P., Ahamad, M., 2017. SoK: Fraud in telephony networks, in: 2017 IEEE European Symposium on Security and Privacy (EuroS&P), IEEE. https://doi.org/10.1109/EuroSP.2017.40.

Siadati, H., Nguyen, T., Gupta, P., Jakobsson, M., Memon, N., 2017. Mind your SMSes: Mitigating social engineering in second factor authentication. Computers & Security 65, 14–28. https://doi.org/10.1016/j.cose.2016.09.009.

Twilio Inc., 2026a. SMS pumping risk score (Lookup v2). https://www.twilio.com/docs/lookup/v2-api/sms-pumping-risk. Accessed 2026-10-01.

Twilio Inc., 2026b. Verify API. https://www.twilio.com/en-us/user-authentication-identity/verify. Accessed 2026-10-03.

Twilio Inc., 2026c. Verify fraud guard. https://www.twilio.com/docs/verify/preventing-toll-fraud/sms-fraud-guard. Accessed 2026-10-01.

Veeravalli, V.V., Banerjee, T., 2012. Quickest change detection. arXiv:1210.5552 (book chapter).

Wald, A., 1945. Sequential tests of statistical hypotheses. The Annals of Mathematical Statistics 16, 117–186. https://doi.org/10.1214/aoms/1177731118.

Web Incubator Community Group, 2026. WebOTP API. https://github.com/WICG/WebOTP. Accessed 2026-10-01.

Wiefling, S., Lo Iacono, L., Dürmuth, M., 2019. Is this really you? An empirical study on risk-based authentication applied in the wild, in: ICT Systems Security and Privacy Protection (IFIP SEC 2019), Springer. pp. 134–148. https://doi.org/10.1007/978-3-030-22312-0_10.

Zaliskyi, M., Odarchenko, M., 2026. Artificially inflated traffic in A2P messaging: Window-based detection models and prefix-level analysis for fraud mitigation. Science-Based Technologies 69, 89–98. https://doi.org/10.18372/2310-5461.69.20735.